DK_ICSOP_7 - B.Sc. in Computing Systems and Operations

DK_ICCSO_8 – B.Sc. (Hons) in Computing Systems and Operations

Stage 3 - Semester 5

JANUARY 2026

COMP I8062 - Algorithms

Internal Examiner(s): Dr. Michelle Graham

External Examiner(s): xxxxxxx

Date:

Time:

Duration:

Venue:

No. Pages:

Instructions

Answer QUESTION ONE (40 marks) and any other TWO questions (30 marks each).

PLEASE NOTE: Exam Paper printed DOUBLE SIDED.

DO NOT TURN THIS PAGE UNTIL INSTRUCTED

You MUST attempt this question.

Question 1) Algorithm Design and Complexity

(a) **Time complexity** and **space complexity** are two key metrics for the analysis of algorithms.

(i) Define time complexity and space complexity in the context of algorithm analysis. Explain why these measures are important when comparing or evaluating algorithms.

(5 marks)

(ii) Explain the difference between **best-case**, **worst-case**, and **average-case** time complexities. Provide an example of an algorithm where the best-case and worst-case complexities differ significantly.

(5 marks)

(iii) Outline two **additional considerations** that can be used to assess algorithms beyond worst-case time complexity. Explain what each adds to the evaluation of an algorithm's performance.

(8 marks)

(b) The algorithm found in appendix A is an implementation of Binary Search.

(i) Write and explain the **code** for an alternative implementation of Binary Search that employs recursion instead of iteration. Indicate and explain the base case and recursive case in your implementation

(16 marks)

(ii) Does the **space** complexity of your implementation of Binary Search differ from that of the original implementation shown in appendix A? Justify your answer with reference to the space complexity of each implementation.

(6 marks)

Choose any TWO questions from questions 2, 3 and 4

Question 2) Sorting Algorithms

(a) **Bubble sort** and **Quick sort** are examples of sorting algorithms using different algorithmic approaches.

(i) Compare the time complexities of these algorithms. Explain under what (if any) conditions one of these algorithms might perform better than the other.

(5 marks)

(ii) What is stability in sorting? Give an example where stability is required.

(5 marks)

(iii) Only one of these algorithms is stable by default. Identify which algorithm is stable, explaining why the stability property applies to one but not the other.

(5 marks)

(b) **Bucket Sort** is a sorting algorithm that distributes elements of an array into several "buckets" and then sorts each bucket individually, typically using another sorting algorithm. Discuss an appropriate sort algorithm to be used on these individual buckets for each of the following considerations:

(i) Small bucket sizes are guaranteed.
(ii) Sort stability is required.
(iii) Data is not uniformly distributed across buckets.

(15 marks)

Question 3) Two-Pointer Technique

(a) Explain the **two-pointer technique**. In what types of problems is this technique commonly used, and why is it effective?

(6 marks)

(b) The algorithm shown in **appendix B** finds the maximum sum that can be generated from a specified-size sequence of values in an array of integers.

(i) How is the sliding-window implementation of the two-pointer technique being used here? Why does this approach remove the need to re-sum the elements of each subarray from scratch?

(6 marks)

(ii) Write and explain the **pseudocode** for a *brute-force* solution to this problem (finding the maximum sum of any contiguous subarray of a specified length).

(10 marks)

(iii) Compare and contrast the time complexity of your brute-force implementation to that of the original two-pointer implementation. In what situations, if any, might the brute-force approach be preferable? Justify your answer.

(8 marks)

Question 4) Selecting Algorithms

(a) You are processing an IoT dataset collected as an array of timestamped events. Due to a sensor malfunction, the data is now unsorted and contains duplicate entries. Rather than discarding the dataset entirely, you have chosen to try and filter out the duplicate information before continuing with the remainder of your processing. This is envisioned as a standalone action, i.e. it does not need to be repeated.

(i) Outline an approach to deduplicating unordered data. What is the time complexity of this process?

(6 marks)

(ii) Deduplicating sorted data may perform with better time complexity than deduplicating unsorted data. Outline an approach to deduplicating sorted data and state its time complexity. Does this approach produce better time complexity than your outlined approach from part (i)? Explain why this is the case.

(12 marks)

(iii) A teammate has suggested sorting the data before carrying out deduplication to allow the use of the sorted deduplication algorithm. Write a short note discussing the comparative performance of taking this approach vs deduplicating the original (unordered) array and recommending an approach. Your discussion should specify the sorting algorithm that would be employed as well as any other relevant details.

(12 marks)

APPENDIX A:

```
349     public static int search(int [] nums, int value){
350         if(nums == null){
351             throw new IllegalArgumentException("Array to be searched cannot be null");
352         }
353
354         int start = 0;
355         int end = nums.length-1;
356
357         while(start <= end) {
358             int mid = start + (end - start) / 2;
359
360             if (nums[mid] == value){
361                 return mid;
362             }else if(value > nums[mid]){
363                 start = mid + 1;
364             }else{
365                 end = mid -1;
366             }
367
368         }
369         return -1;
370     }
```

APPENDIX B:

```java
public static int maxSum(int arr[], int numElements){
    if(arr == null || arr.length == 0){
        throw new IllegalArgumentException("Cannot find max sum in null or empty array");
    }

    if(numElements > arr.length){
        throw new IllegalArgumentException("Number of elements to be included in sum must not
            exceed length of array");
    }

    int maxSum = 0;
    for (int i = 0; i < numElements; i++) {
        maxSum += arr[i];
    }

    int windowSum = maxSum;
    for (int i = numElements; i < arr.length; i++) {
        windowSum += arr[i];
        windowSum -= arr[i-numElements];

        maxSum = Math.max(windowSum, maxSum);
    }

    return maxSum;
}
```